



Sri Lanka Institute of Information Technology

SE3090 – Software Engineering Frameworks
Year 3, Semester 1- 2026

Practical Answer Submission

Practical Sheet No: 01

Student ID	IT24100559
Student Name	Wasala W.M.S.S.B
Campus/ Center name	Malabe
Specialization	SE
Batch No	Y03S01.WE.SE.0101

Task 01 — Framework or Library? Classify and Justify

#	Technology	Framework / Library	Justification (who controls the flow?)
1	lodash	Library	My code calls lodash utility functions (e.g. <code>_.map</code>) whenever it chooses; lodash never calls my code.
2	ASP.NET Core	Framework	The host and middleware pipeline own the request lifecycle and invoke my controllers at defined extension points.
3	axios	Library	My code explicitly calls <code>axios.get()</code> ; axios has no lifecycle and does not drive execution.
4	Flutter	Framework	Flutter runs the widget and render lifecycle and calls my <code>build()</code> method; I never call the render loop myself.
5	jQuery	Library	My code calls <code>\$(...)</code> selectors and methods on demand; jQuery does not control application flow.
6	Angular	Framework	Angular bootstraps the application, manages dependency injection and change detection, and calls my component lifecycle hooks such as <code>ngOnInit</code> .
7	Express.js	Framework (minimal)	Express owns the HTTP listener and routing loop and invokes my route handlers and middleware when requests arrive.
8	NumPy	Library	My code calls <code>np.array()</code> and array operations; NumPy provides functionality but no control flow.
9	React ★	Framework (grey zone — defended below)	React's reconciler owns the render cycle and calls my components and Hooks; I do not call the renderer.
10	Spring Boot	Framework	The Spring container manages the bean lifecycle, auto-configuration and dependency injection, and calls my annotated classes.

★ React — Grey-Zone Defence

React calls itself a UI library because it mainly covers the view layer and depends on separate packages for routing and state management. However, its lifecycle controls when components render, so under the Inversion of Control test, it can be defended as a framework.

Task 02 — Framework Comparison Table

Framework family chosen: Frontend (Web UI)

Criterion	Framework A: React	Framework B: Angular	Framework C: Vue
Type	Component-based UI framework / library covering the view layer	Full, opinionated single-page application framework	Progressive framework, incrementally adoptable
Language	JavaScript / TypeScript (JSX)	TypeScript (mandatory)	JavaScript / TypeScript
Best fit / strengths	Flexible SPAs, reusable components and a large ecosystem.	Large enterprise applications requiring built-in structure and services.	Lightweight applications, rapid prototyping and easy onboarding.
Used by (industry)	Meta, Netflix, Airbnb	Google, Microsoft Office web	Alibaba, GitLab
Main trade-off / cost	Not batteries-included — routing and state management must be chosen and assembled separately	Steepest learning curve; heavier and more verbose	Smaller ecosystem and smaller hiring pool than React

Verdict — 3 developers, 4 months, student startup

For a 3-developer startup with a 4-month deadline, I would choose React. Its large ecosystem, reusable components and strong community reduce development time and help solve problems quickly. Angular provides more built-in structure but has a steeper learning curve, while Vue is easier to learn but has a smaller hiring pool. Therefore, React offers the best balance of speed, ecosystem and future growth.

Task 03 — Selection Workflow, Weighted Matrix and Stack Plan

Scenario: GreenCart - Campus Grocery Pickup

Selection workflow: Requirements -> Constraints -> Shortlist -> Prototype -> Decide & record.

Key constraints: 3 developers, 4 months and a small budget.

Part A - Weighted Decision Matrix

Layer evaluated: Mobile

Shortlisted candidates: Flutter, React Native and Native (SwiftUI + Jetpack Compose).

Assumption: The team has moderate experience with Flutter and React Native but limited experience maintaining separate native iOS and Android applications.

Scoring scale: 1 = poor and 5 = excellent.

Criterion	Weight %	Flutter	React Native	Native
Team familiarity	20%	4	4	2
Time to first working version	30%	5	4	2
Hiring pool (if team grows)	15%	3	4	4
Ecosystem for our features	20%	4	4	5
Long-term maintainability	15%	4	3	5
Weighted total	100%	4.15	3.85	3.25

Weighted Total Calculations

Flutter = $(0.20 \times 4) + (0.30 \times 5) + (0.15 \times 3) + (0.20 \times 4) + (0.15 \times 4) = 4.15$

React Native = $(0.20 \times 4) + (0.30 \times 4) + (0.15 \times 4) + (0.20 \times 4) + (0.15 \times 3) = 3.85$

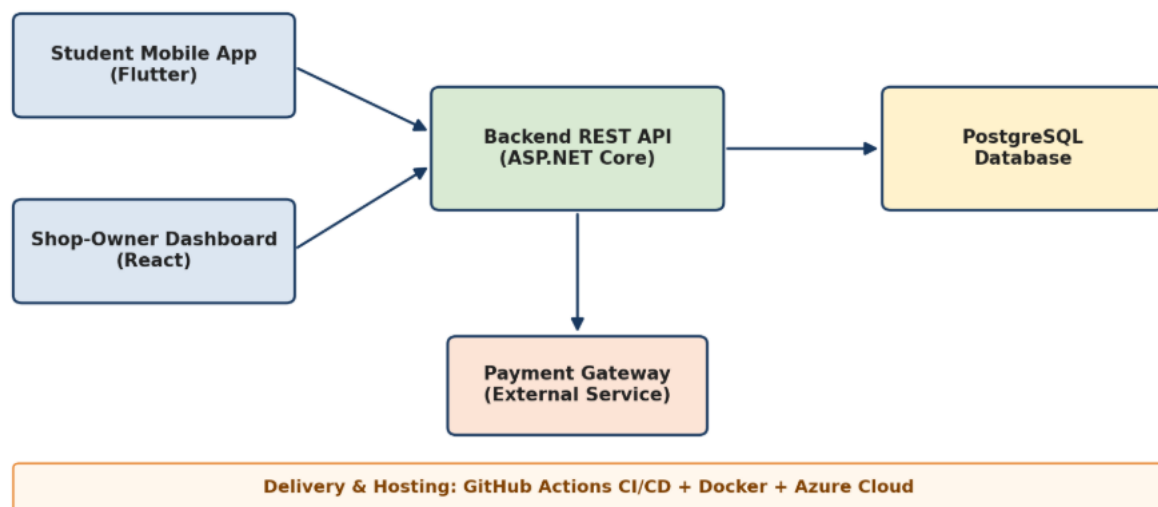
Native = $(0.20 \times 2) + (0.30 \times 2) + (0.15 \times 4) + (0.20 \times 5) + (0.15 \times 5) = 3.35$

Winner: Flutter, with the highest weighted total of 4.15. Time to the first working version received the highest weight because the team must deliver within four months.

Part B — GreenCart Technology Stack Plan

Layer	Your choice	Justification (cite one criterion)
Mobile app	Flutter	Project fit — a single codebase delivers both iOS and Android, and a 3-developer team cannot staff two separate native tracks within 4 months.
Web dashboard	React	Team skills and ecosystem — the largest component ecosystem lets a small team assemble the shop-owner dashboard (tables, charts, forms) quickly.
Backend API	ASP.NET Core	Maturity and performance — a mature, well-documented framework with built-in authentication, dependency injection and a middleware pipeline that scales as more faculties are added.
Database	PostgreSQL	Project fit and cost — orders and payments require relational integrity and ACID transactions, and being open-source keeps the cost within a small budget.
Delivery and hosting	GitHub Actions + Docker + Azure	Cost and scalability - automated delivery and managed cloud hosting reduce administration and support future growth.

Part C — Architecture Sketch



Task 04 — Real-World Case and Emerging-Trends Reflection

Scenario chosen: Option B — Nationwide bus company real-time seat-booking and live-tracking platform with heavy, spiky holiday traffic.

Part A — Select and Justify a Stack

Two most important non-functional requirements

- Scalability / elasticity — booking traffic rises sharply during holiday seasons and must be absorbed without downtime.
- Reliability and consistency — double-booking of a seat must never occur, so seat reservation must be transactionally safe.

Top three selection criteria

#	Criterion	Why it matters in this scenario
1	Performance and scalability	Traffic is spiky; the system must handle holiday peaks without slow or failed seat locking.
2	Ecosystem for real-time features	Live tracking requires mature WebSocket, push-notification and mapping integrations rather than custom protocol work.
3	Maturity and reliability	Payment and seat-reservation consistency demands a proven, battle-tested stack, not an experimental one.

Stack selection

Layer	Choice	Justification (criterion)
Mobile (passenger app)	Flutter	Project fit — one codebase covers iOS and Android, which is essential for nationwide reach from day one.
Web (operator portal)	React	Ecosystem — mature mapping and dashboard libraries support live fleet monitoring.
Backend API	ASP.NET Core	Performance and maturity — high-throughput asynchronous request handling with proven production reliability.
Real-time layer	WebSockets (SignalR)	Ecosystem — first-class real-time support for live vehicle positions and seat-lock updates.
Database	PostgreSQL with Redis cache	Reliability — PostgreSQL provides ACID transactions that prevent double-booking; Redis caches seat availability under peak load.
Hosting	Cloud platform with auto-scaling and CI/CD	Scalability — elastic capacity matches spiky seasonal demand instead of paying for permanent peak-capacity hardware.

Part B — Emerging-Trends Reflection

Trend	Apply?	Where / why	One risk or cost
Microservices	Yes (partial)	Split only booking, live tracking and payments, because booking must scale independently during holiday peaks while tracking carries a constant real-time load.	Operational complexity — distributed transactions, network latency and the need for a mature DevOps capability; a full split would over-engineer the first release.
Serverless functions	Yes (targeted)	Used for bursty, stateless side-tasks such as ticket email/SMS notifications, receipt generation and scheduled reports, which spike together with bookings.	Cold-start latency makes serverless unsuitable for the seat-locking path, and costs become unpredictable at very high sustained volume.
Cloud-native and DevOps (CI/CD)	Yes	Essential — auto-scaling absorbs holiday traffic, and automated build, test and deploy pipelines allow safe fixes during peak season.	Initial setup time and monitoring/tooling cost; a misconfigured auto-scaling policy can generate large unexpected cloud bills.
AI-assisted / agentic development	Yes (with human oversight)	Speeds up boilerplate, test generation and documentation, letting a small team concentrate on booking-consistency logic.	Generated code may contain subtle security or concurrency defects; the human engineer must review, test and remain responsible for the output.

Reflection

Framework options change, but the Requirements -> Constraints -> Shortlist -> Prototype -> Decide & record method remains reusable.

Quick Test — Knowledge Check (Q1–Q8)

Multiple choice (choose one)

Q1. The defining difference between a framework and a library is:

- (a) size
- (b) programming language
- (c) who controls the flow of execution
- (d) the license

Answer: (c) who controls the flow of execution

Q2. "Don't call us, we'll call you" (the Hollywood Principle) describes:

- (a) REST
- (b) Inversion of Control
- (c) CI/CD
- (d) serverless computing

Answer: (b) Inversion of Control

Q3. A 3-person startup needs iOS + Android apps in 4 months. The strongest first candidate is:

- (a) two separate native apps
- (b) Flutter (cross-platform)
- (c) a desktop application
- (d) a static website

Answer: (b) Flutter (cross-platform)

Q4. The best reason to choose framework X over framework Y is that X:

- (a) is trending on social media
- (b) is used by Google
- (c) has the highest weighted score against your project's criteria
- (d) had the newest release this week

Answer: (c) has the highest weighted score against your project's criteria

Q5. In agentic / AI-assisted development, the human engineer's essential role is to:

- (a) type faster
- (b) review, test and take responsibility for the output
- (c) avoid writing documentation
- (d) remove all human oversight

Answer: (b) review, test and take responsibility for the output

Short answer (1–2 sentences each)

Q6. Give one benefit AND one cost of adopting a large framework for a short student project.

Benefit: A large framework provides built-in structure and reusable services. Cost: Its learning curve and configuration overhead may delay a short project.

Q7. Name the five stages of the framework selection workflow taught in the lecture, in order.

Requirements → Constraints → Shortlist → Prototype → Decide & Record.

Scenario-based

Q8. A classmate says: "I imported axios into my project, so now I'm using a framework." Is this correct? Justify your answer in one or two sentences using the control-of-flow test.

No. Axios is a library because the developer calls it when needed; it does not control the application lifecycle or invoke the developer's code through Inversion of Control.